

Sessão 2 - Tratamento de dados

Treinamento em R - SEFAZ RS-WB DIME

Thiago Scot

The World Bank | Fevereiro 2025

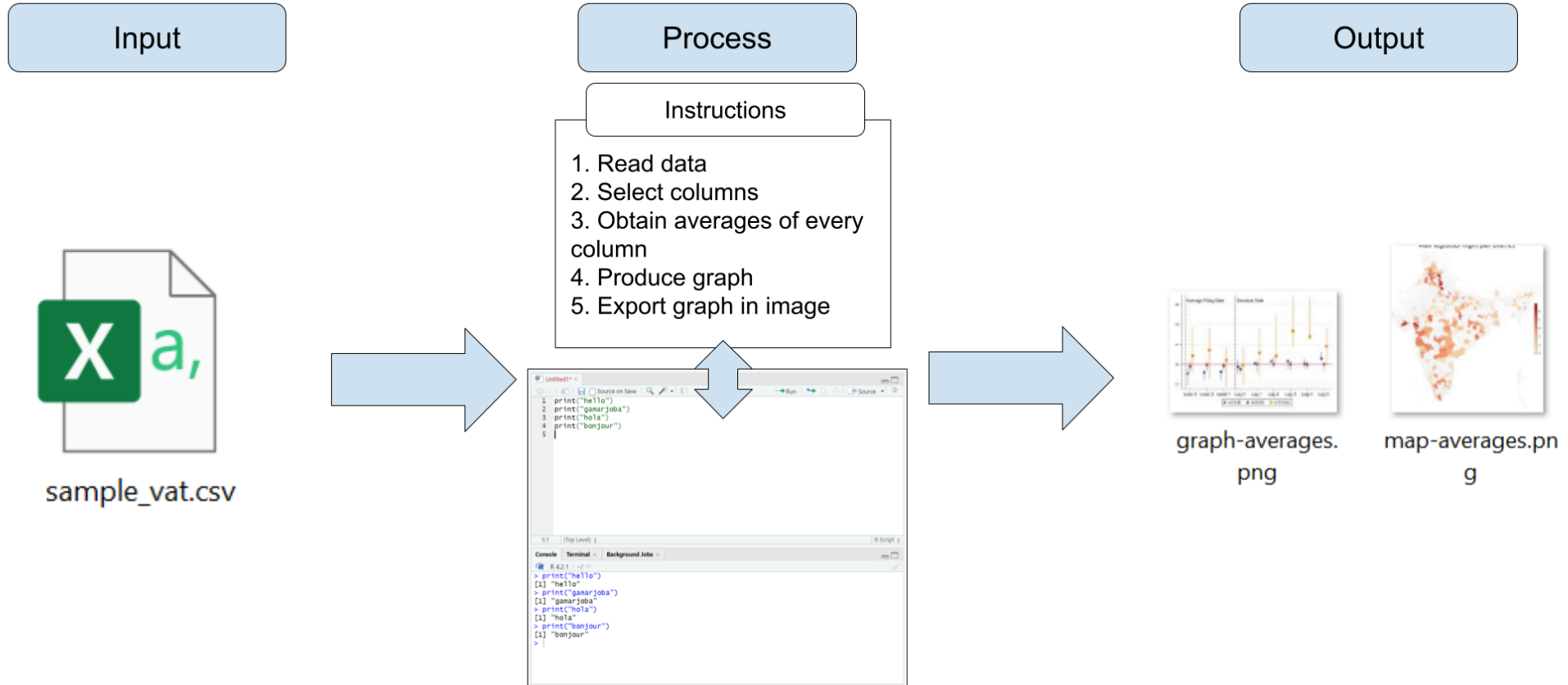


Índice

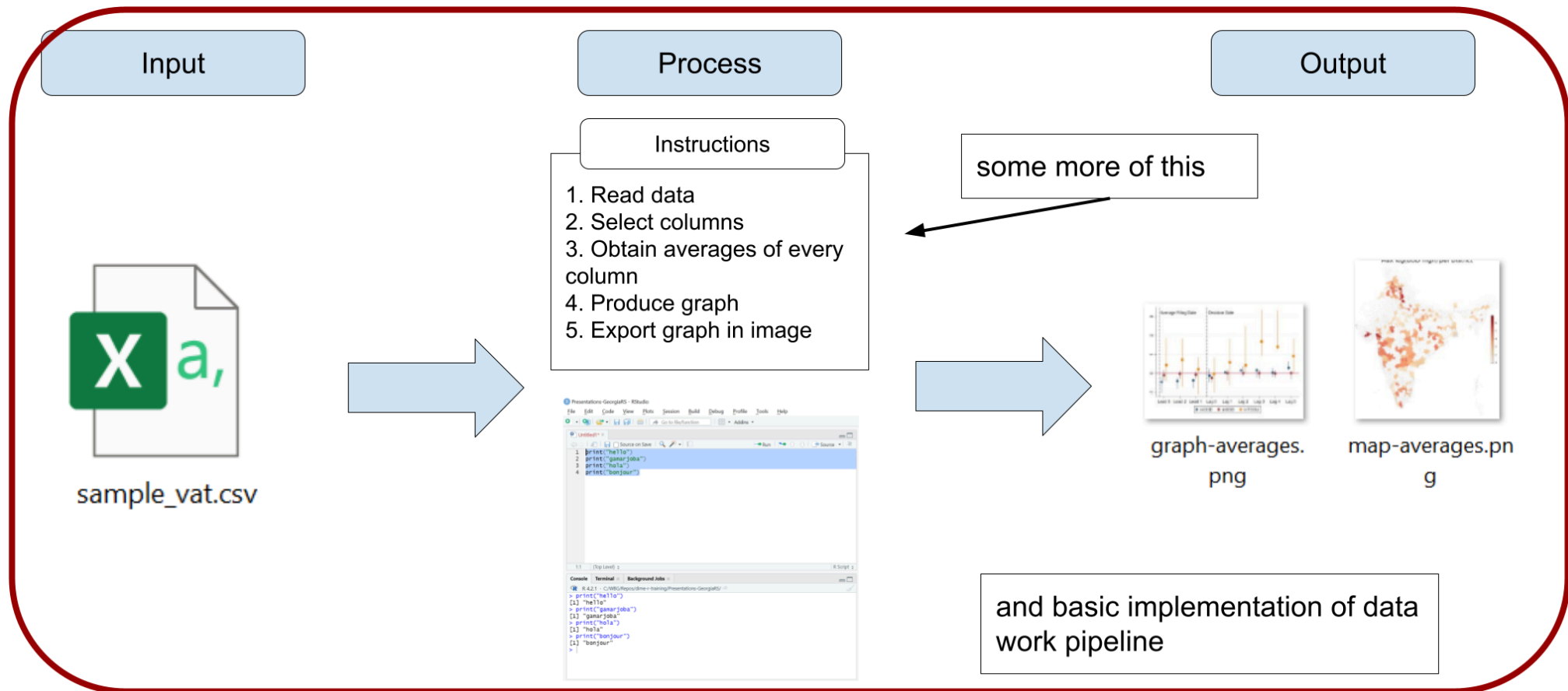
1. Sobre esta sessão
2. Bibliotecas R
3. Manipulação de dados
4. Filtragem e ordenação
5. Junção de dataframes
6. Exportação de resultados
7. Conclusão
8. Apêndice

Sobre esta sessão

Sobre esta sessão



Sobre esta sessão



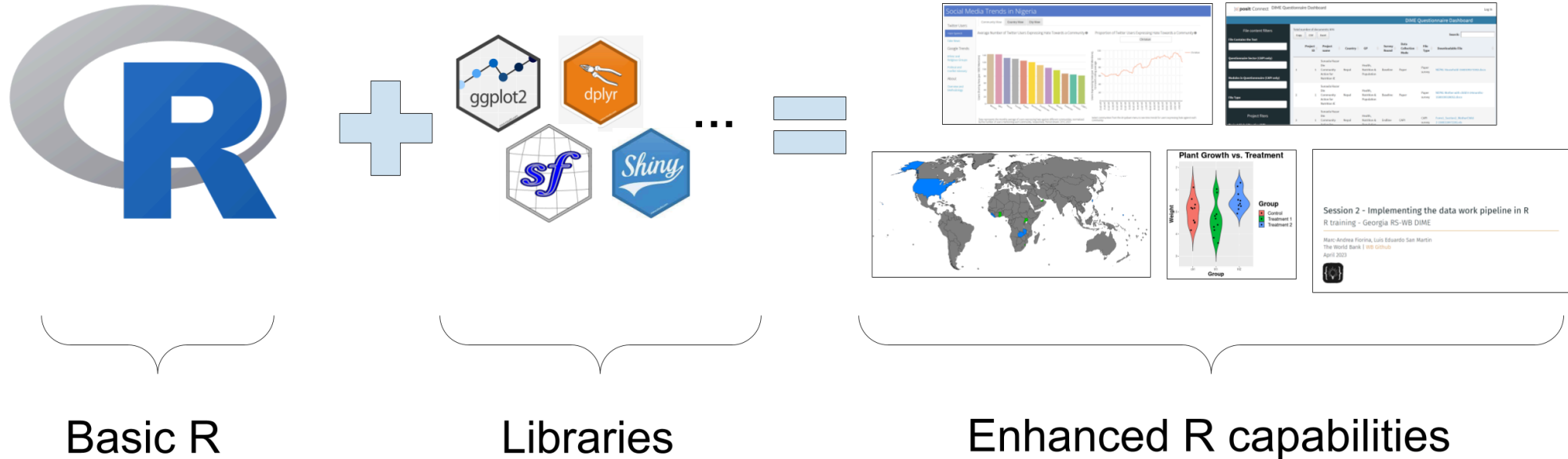
Bibliotecas R

Bibliotecas R

- Instalar o R no seu computador fornece acesso às suas funções básicas.
- Além disso, você pode instalar bibliotecas. Bibliotecas são pacotes de funções adicionais do R que permitem:
 - Executar operações que as funções básicas do R não fazem (exemplo: trabalhar com dados geográficos)
- Realizar operações que o R básico faz, mas de forma mais fácil (exemplo: manipulação de dados)

Bibliotecas R

Em resumo:

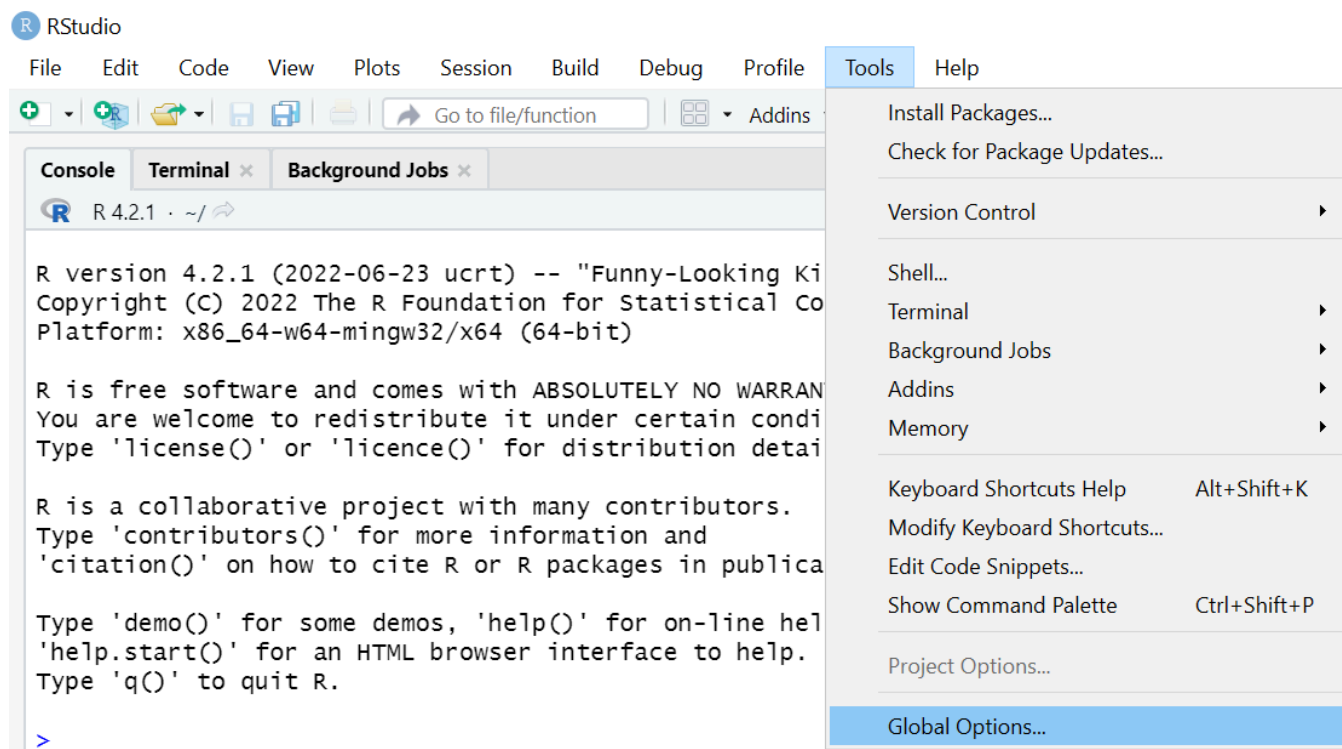


Instalando bibliotecas R

- A instalação de bibliotecas geralmente é simples, mas pode ser desafiadora em conexões institucionais, como no Banco Mundial ou na SEFAZ.
- O próximo exercício configurará o RStudio para instalar bibliotecas sem problemas.
- Nota: os pacotes já devem estar instalados nas suas máquinas

Exercício 1: Configurar a instalação de bibliotecas

1. No RStudio, vá para **Tools** >> **Global Options...**



Bibliotecas R

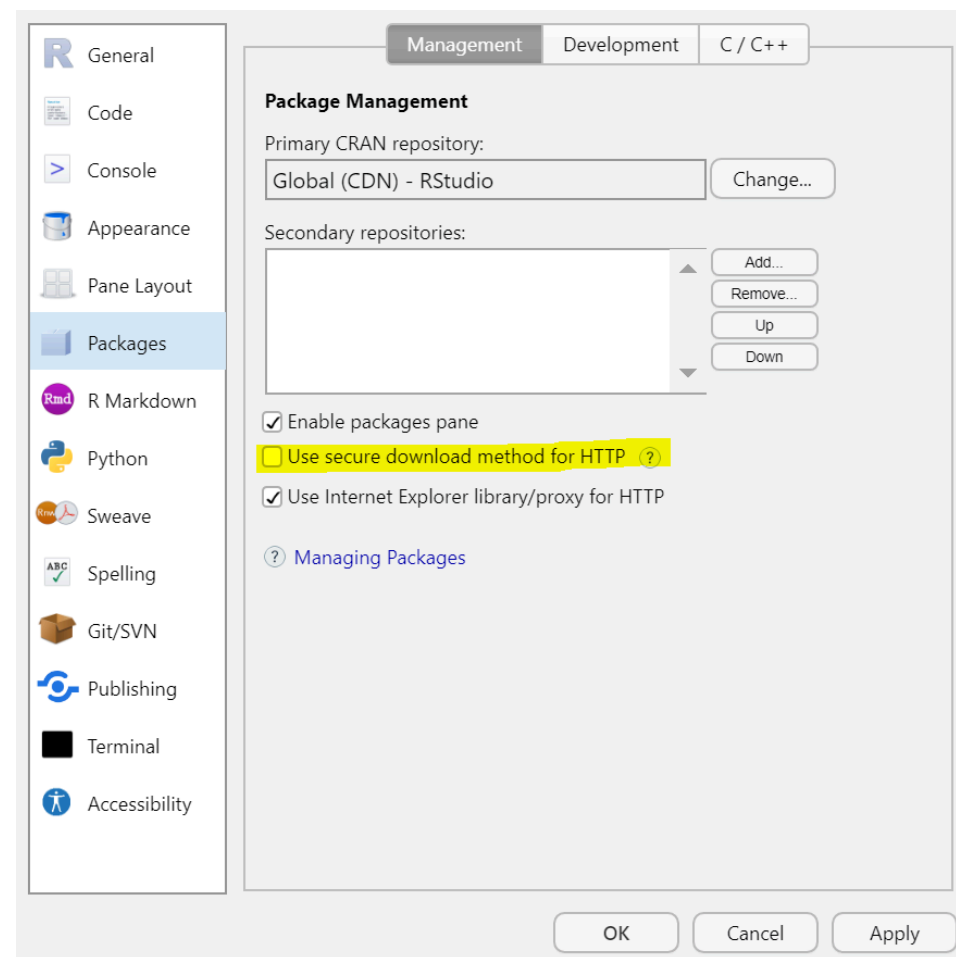
Exercício 1: Configurar a instalação de bibliotecas

1. Selecione **Packages** no painel esquerdo.

1. Desmarque **Use secure download method for HTTP**.

2. Clique em **OK**.

Você não verá nenhuma mudança na janela do RStudio, mas agora será possível instalar bibliotecas sem problemas.



Bibliotecas R

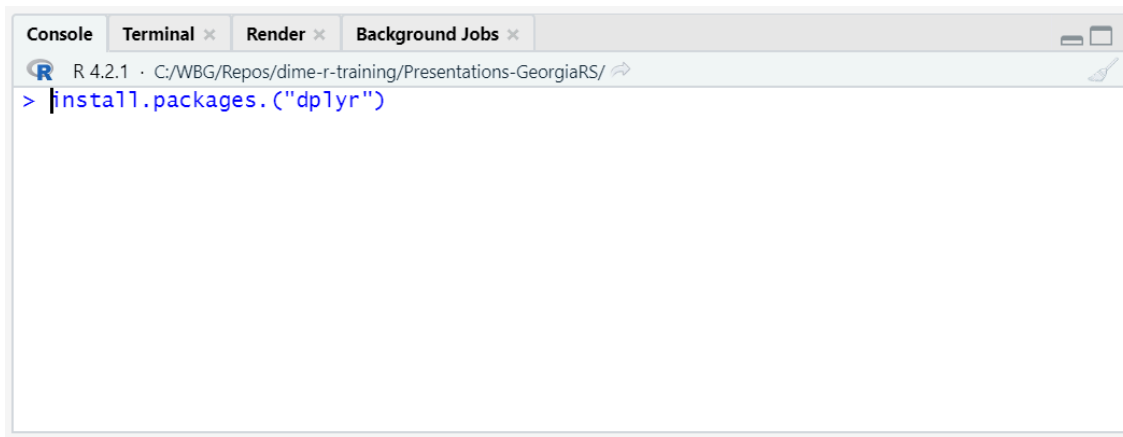
Utilizaremos a biblioteca `dplyr` na sessão de hoje.

Exercício 2: Instalando bibliotecas

1. Instale a biblioteca com:

```
install.packages("dplyr")
```

- Note o uso de aspas (" ") nos nomes dos pacotes.
- **Digite este código no console**, não no script.



Bibliotecas R

Agora que `dplyr` está instalado, precisamos carregá-lo para usar suas funções.

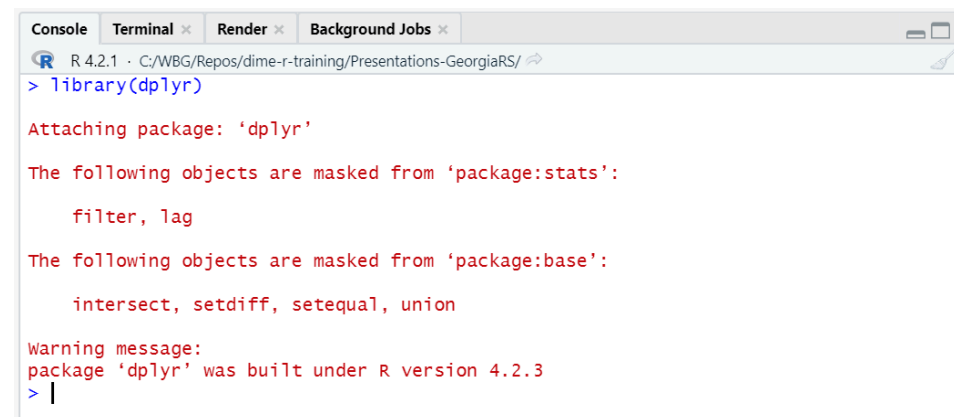
Exercício 3: Carregar bibliotecas

1. Abra um novo script com `File >> New File >> R Script`.

1. Carregue `dplyr` com:

```
library(dplyr)
```

- Execute este código no novo script aberto.
- Note que **não utilizamos aspas** nos nomes das bibliotecas ao carregá-las.



```
Console Terminal x Render x Background Jobs x
R 4.2.1 · C:/WBG/Repos/dime-r-training/Presentations-GeorgiaRS/
> library(dplyr)

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

  filter, lag

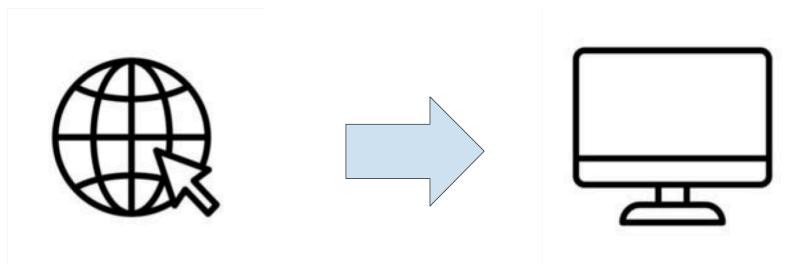
The following objects are masked from 'package:base':

  intersect, setdiff, setequal, union

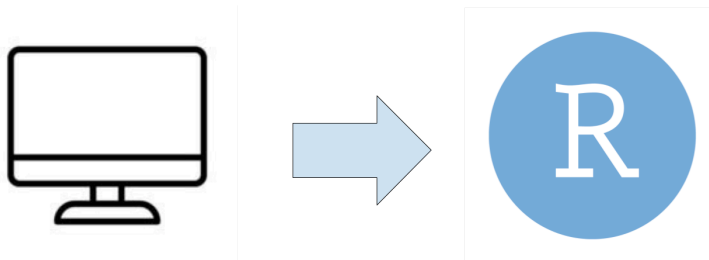
Warning message:
package 'dplyr' was built under R version 4.2.3
> |
```

Bibliotecas R // Bibliotecas R

- Instalação da biblioteca:



- Carregamento da biblioteca:



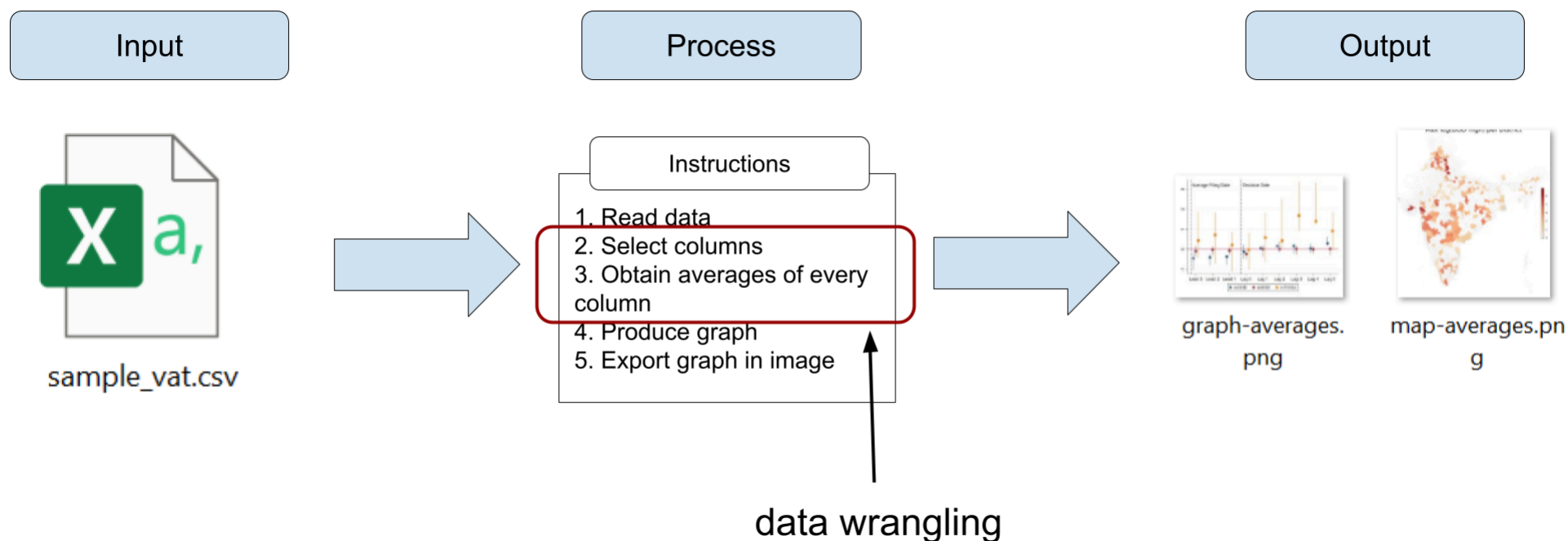
- Você instala as bibliotecas do R apenas uma vez no seu computador
- Você carrega as bibliotecas sempre que abre uma nova janela do RStudio (somente carregue as bibliotecas que usará)

Manipulação de dados // Data Wrangling

Manipulação de dados // Data Wrangling

Preparando seus dados

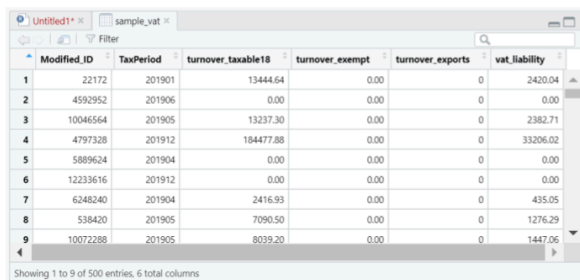
- Os dados raramente estão em um formato pronto para serem convertidos diretamente em um resultado
- Em programação estatística, o processo de transformar dados em uma condição pronta para ser convertida em um resultado é chamado de **manipulação de dados**



Manipulação de dados // Data Wrangling

Preparando seus dados

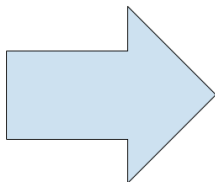
- A manipulação de dados é um dos aspectos mais cruciais e demorados do trabalho com dados
- Envolve não apenas a codificação, mas também o exercício mental de pensar qual deve ser a estrutura e condição do seu dataframe para produzir o resultado desejado



Showing 1 to 9 of 500 entries, 6 total columns

	Modified_ID	TaxPeriod	turnover_taxable18	turnover_exempt	turnover_exports	vat_liability
1	22172	201901	13444.64	0.00	0	2420.04
2	4592952	201906	0.00	0.00	0	0.00
3	10046564	201905	13237.30	0.00	0	2382.71
4	4797328	201912	184477.88	0.00	0	33206.02
5	5889624	201904	0.00	0.00	0	0.00
6	12233616	201912	0.00	0.00	0	0.00
7	6248240	201904	2416.93	0.00	0	435.05
8	538420	201905	7090.50	0.00	0	1276.29
9	10072288	201905	8039.20	0.00	0	1447.06

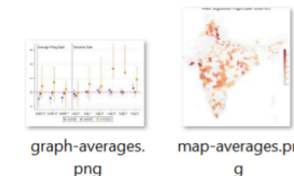
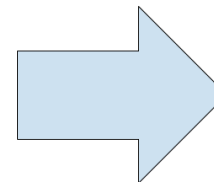
initial dataframe



Showing 1 to 9 of 500 entries, 6 total columns

	Modified_ID	Tax	turnover_exempt	turnover_exports	vat_liability
1	22172		0.00	0	2420.04
2	4592952		0.00	0	0.00
3	10046564		0.00	0	2382.71
4	4797328		0.00	0	33206.02
5	5889624		0.00	0	0.00
6	12233616		0.00	0	0.00
7	6248240		0.00	0	435.05
8	538420	201905	7090.50	0	1276.29
9	10072288	201905	8039.20	0.00	1447.06

dataframe needed for outputs



outputs

Manipulação de dados // Data Wrangling

Preparando seus dados

- Como mencionamos antes, usaremos `dplyr` e `tidyr` para manipulação de dados neste treinamento
 - Você também pode usar R básico, mas recomendamos essas bibliotecas porque suas funções são mais fáceis de usar
 - Num próximo passo, recomendo uso de `data.table` como ferramenta poderosa de análise de grandes bases



Manipulação de dados // Data Wrangling

Exercício 4: Carregando dados

Esta parte é igual ao exercício que fizemos na sessão 1, mas não há problema em repeti-la para iniciar uma nova sessão do RStudio. **Se você tem o RStudio aberto, feche a janela e abra o RStudio novamente.**

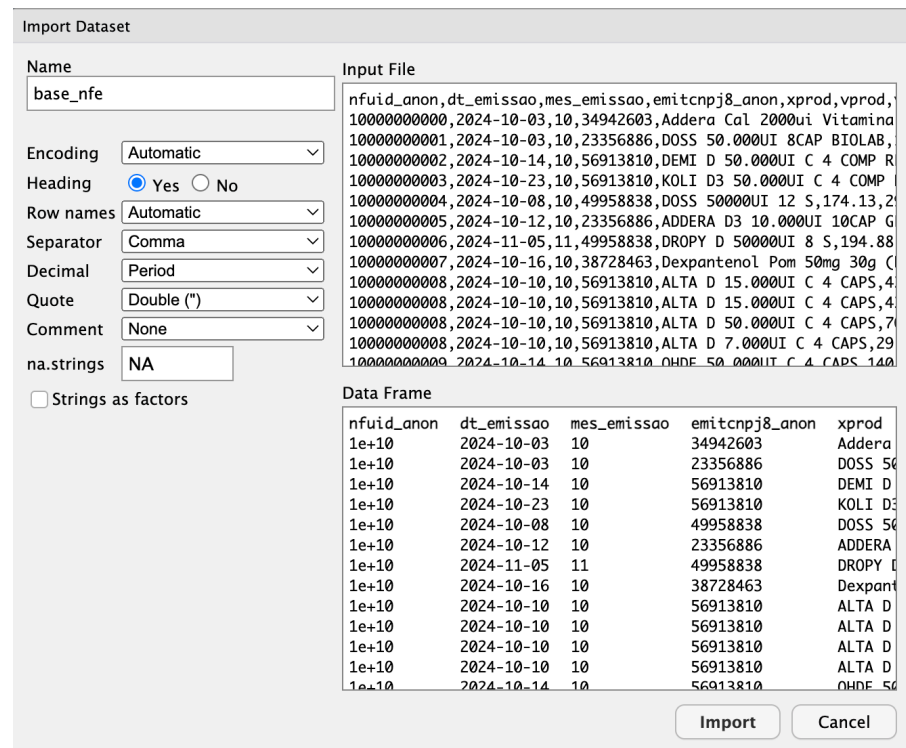
1. Na sua nova janela do RStudio, vá para **File** > **Import Dataset** > **From Text (base)** e selecione novamente o arquivo **base_nfe.csv**

- se você não sabe onde o arquivo está, verifique a pasta **Downloads**

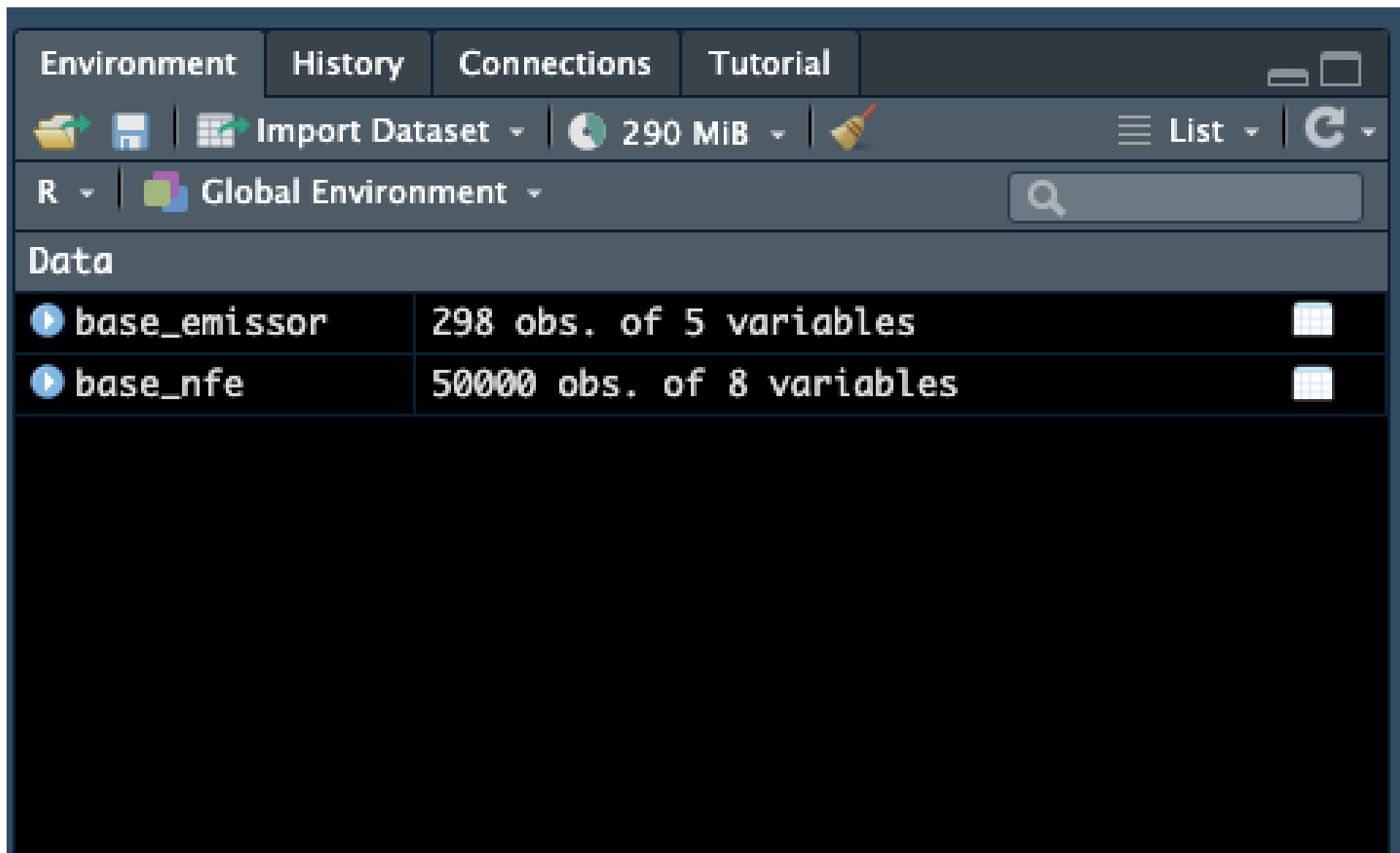
1. Certifique-se de selecionar **Heading** > **Yes** na próxima janela

2. Selecione **Import**

3. Faça o mesmo para o banco de dados **base_emissor.csv**



Manipulação de dados // Data Wrangling



The screenshot shows the RStudio Environment pane. At the top, there are tabs for 'Environment', 'History', 'Connections', and 'Tutorial'. Below the tabs is a toolbar with icons for file operations and a status bar showing '290 MiB'. The main area of the pane is titled 'Global Environment' and contains a search bar. Below this, a section titled 'Data' lists the objects currently in the environment:

Object	Description	Icon
base_emissor	298 obs. of 5 variables	
base_nfe	50000 obs. of 8 variables	

Manipulação de dados // Data Wrangling

Nota: carregando dados com uma função

- Você também pode carregar dados CSV com a função `read.csv()` ou `data.table::fread()` em vez de usar esse método manual
- O primeiro argumento de `read.csv()` ou `data.table::fread()` é o caminho no seu computador onde os dados estão localizados. Por exemplo:

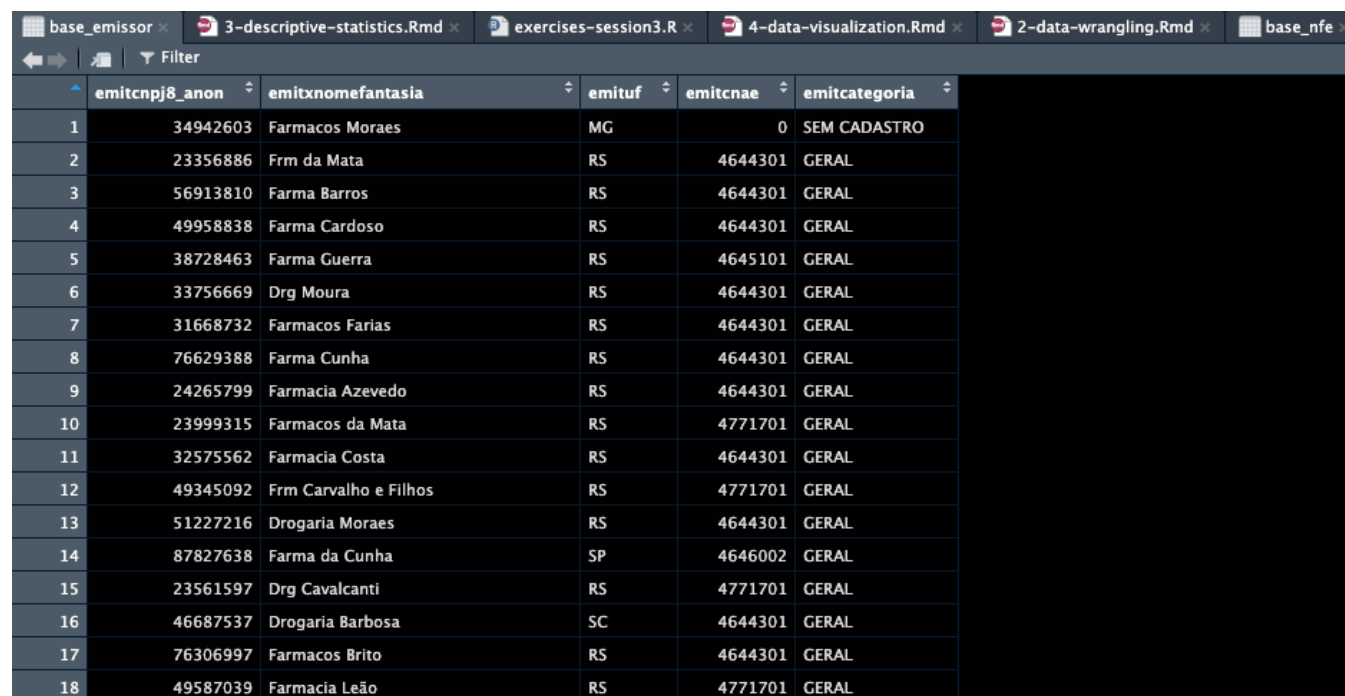
```
base_emissor <- read.csv("/Users/tscot/Downloads/Treinamento_SEFAZ/base_emissor.csv")  
base_emissor <- fread('/Users/tscot/Downloads/Treinamento_SEFAZ/base_emissor.csv')
```

- Como sempre, você precisa armazenar o resultado da função em um objeto dataframe com o operador `<-` ou `=` para que ele seja salvo no ambiente

Manipulação de dados // Data Wrangling

Recapitulando: Conhecendo seus dados

- O dataframe `base_nfe` é o mesmo da sessão anterior, contendo informações ao nível de item da NFe
- O dataframe `base_emissor` é um dataframe no nível do emissor
- A coluna `emitcnpj8_anon` é um identificador anônimo do emissor e `emitxnomefantasia` é um nome aleatório
- A coluna `emituf` indica a unidade federativa do emissor



	emitcnpj8_anon	emitxnomefantasia	emituf	emitcnae	emitcategoria
1	34942603	Farmacos Moraes	MG	0	SEM CADASTRO
2	23356886	Frm da Mata	RS	4644301	GERAL
3	56913810	Farma Barros	RS	4644301	GERAL
4	49958838	Farma Cardoso	RS	4644301	GERAL
5	38728463	Farma Guerra	RS	4645101	GERAL
6	33756669	Drg Moura	RS	4644301	GERAL
7	31668732	Farmacos Farias	RS	4644301	GERAL
8	76629388	Farma Cunha	RS	4644301	GERAL
9	24265799	Farmacia Azevedo	RS	4644301	GERAL
10	23999315	Farmacos da Mata	RS	4771701	GERAL
11	32575562	Farmacia Costa	RS	4644301	GERAL
12	49345092	Frm Carvalho e Filhos	RS	4771701	GERAL
13	51227216	Drogaria Moraes	RS	4644301	GERAL
14	87827638	Farma da Cunha	SP	4646002	GERAL
15	23561597	Drg Cavalcanti	RS	4771701	GERAL
16	46687537	Drogaria Barbosa	SC	4644301	GERAL
17	76306997	Farmacos Brito	RS	4644301	GERAL
18	49587039	Farmacia Leão	RS	4771701	GERAL

Manipulação de dados // Data Wrangling

- Usaremos este dataframe apenas em um dos próximos exercícios, mas o carregamos agora porque é uma boa prática ter os dados carregados na memória para pronto uso
- Nos próximos exercícios, enfrentaremos cenários que nos mostrarão operações de manipulação de dados necessárias

Filtragem e classificação

Solicitação de trabalho com dados

Cenário 1: Imagine que você recebe a seguinte solicitação:

"A Receita Estadual está investigando valores inconsistentes em NFes em dezembro. Gostaríamos que você extraísse os 50 maiores valores de itens para NFes emitidas em dezembro - você poderia produzir essa lista?"

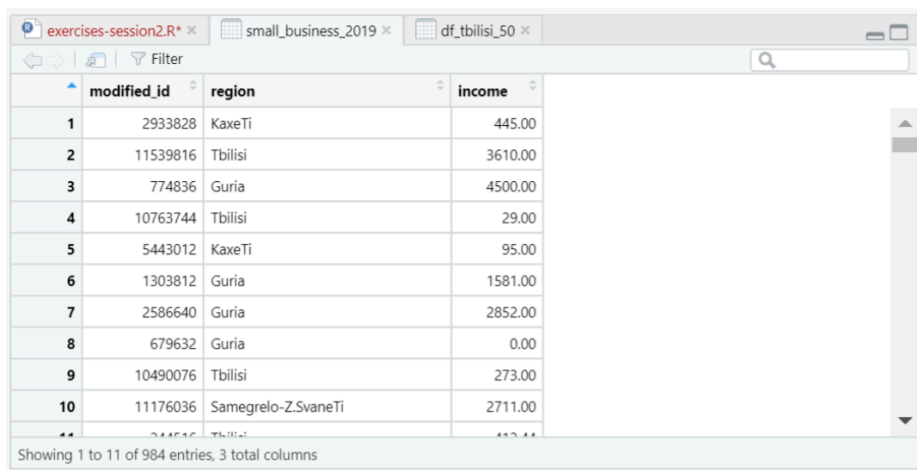
Filtragem e Classificação

Filtragem e Classificação

Solicitação de Trabalho com Dados

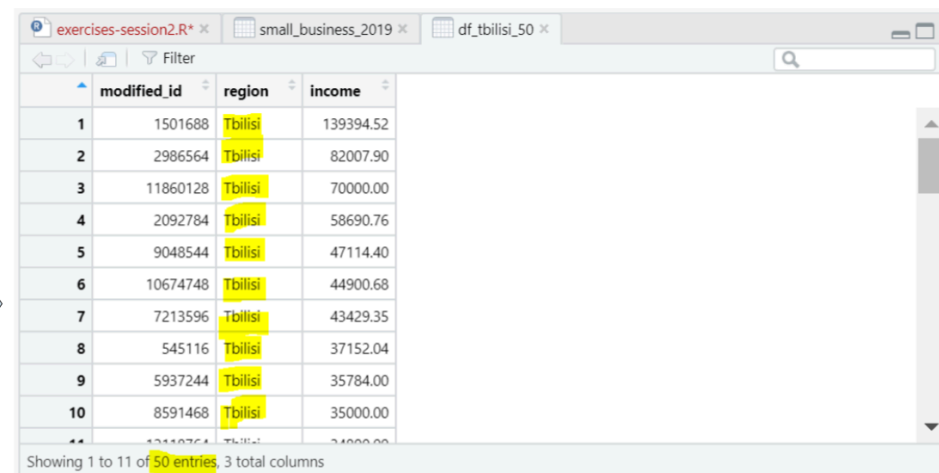
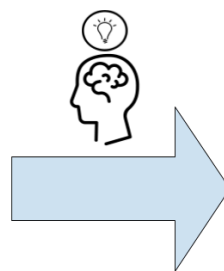
A manipulação dos dados aqui envolve várias operações:

1. Manter apenas as NFes de dezembro
2. Ordenar pelos valores dos itens
3. Manter apenas os 50 primeiros itens



	modified_id	region	income
1	2933828	KaxeTi	445.00
2	11539816	Tbilisi	3610.00
3	774836	Guria	4500.00
4	10763744	Tbilisi	29.00
5	5443012	KaxeTi	95.00
6	1303812	Guria	1581.00
7	2586640	Guria	2852.00
8	679632	Guria	0.00
9	10490076	Tbilisi	273.00
10	11176036	Samegrelo-Z.SvaneTi	2711.00

Showing 1 to 11 of 984 entries, 3 total columns



	modified_id	region	income
1	1501688	Tbilisi	139394.52
2	2986564	Tbilisi	82007.90
3	11860128	Tbilisi	70000.00
4	2092784	Tbilisi	58690.76
5	9048544	Tbilisi	47114.40
6	10674748	Tbilisi	44900.68
7	7213596	Tbilisi	43429.35
8	545116	Tbilisi	37152.04
9	5937244	Tbilisi	35784.00
10	8591468	Tbilisi	35000.00

Showing 1 to 11 of 50 entries, 3 total columns

- Observations: Business reported income of 2019

- Observations: Business reported income of 2019, only for Tbilisi, and the 50 highest

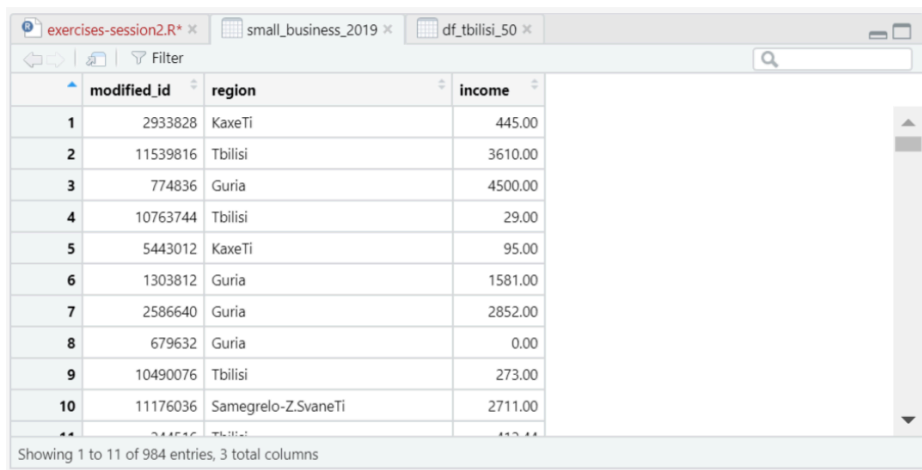
- Columns: notice that the columns are the same as in the starting dataframe

Filtragem e Classificação

1. Manter apenas as NFes de dezembro

Use `filter()` para isso:

```
temp1 <- filter(base_nfe, mes_emissao == 12)
```

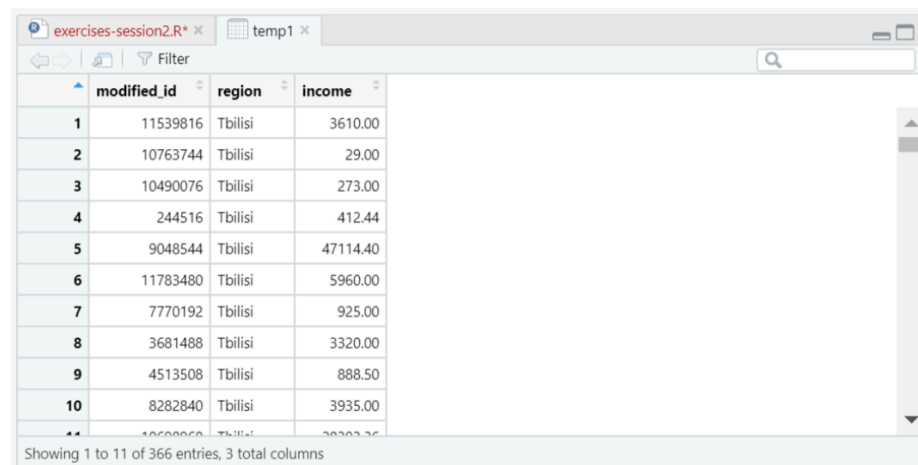
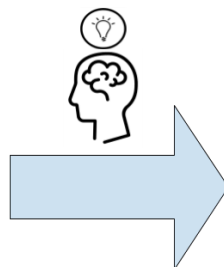


The RStudio interface shows a data frame with 984 entries. The columns are 'modified_id', 'region', and 'income'. The first 10 rows are visible.

	modified_id	region	income
1	2933828	KaxeTi	445.00
2	11539816	Tbilisi	3610.00
3	774836	Guria	4500.00
4	10763744	Tbilisi	29.00
5	5443012	KaxeTi	95.00
6	1303812	Guria	1581.00
7	2586640	Guria	2852.00
8	679632	Guria	0.00
9	10490076	Tbilisi	273.00
10	11176036	Samegrelo-ZsveneTi	2711.00

Showing 1 to 11 of 984 entries, 3 total columns

- Observations: Business reported income of 2019



The RStudio interface shows a filtered data frame with 366 entries. The columns are 'modified_id', 'region', and 'income'. The first 10 rows are visible.

	modified_id	region	income
1	11539816	Tbilisi	3610.00
2	10763744	Tbilisi	29.00
3	10490076	Tbilisi	273.00
4	244516	Tbilisi	412.44
5	9048544	Tbilisi	47114.40
6	11783480	Tbilisi	5960.00
7	7770192	Tbilisi	925.00
8	3681488	Tbilisi	3320.00
9	4513508	Tbilisi	888.50
10	8282840	Tbilisi	3935.00

Showing 1 to 11 of 366 entries, 3 total columns

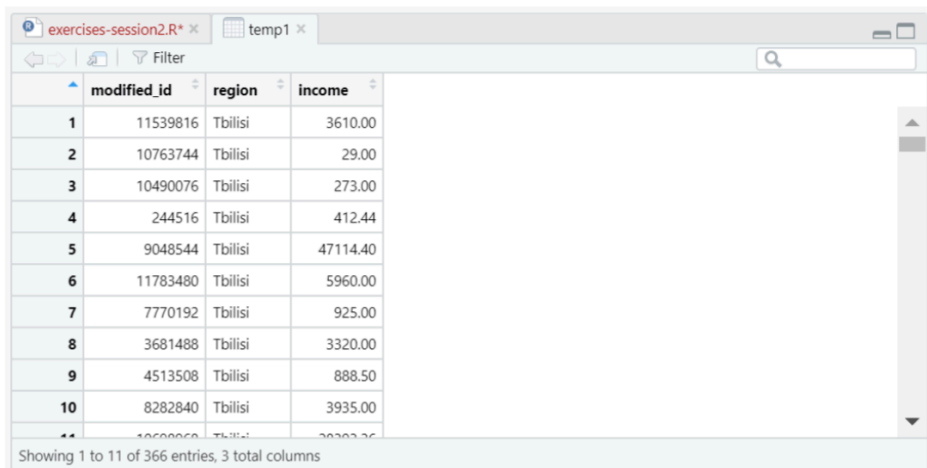
- Observations: Business reported income of 2019, only for Tbilisi

Filtragem e Classificação

2. Ordenar pelos valores dos itens

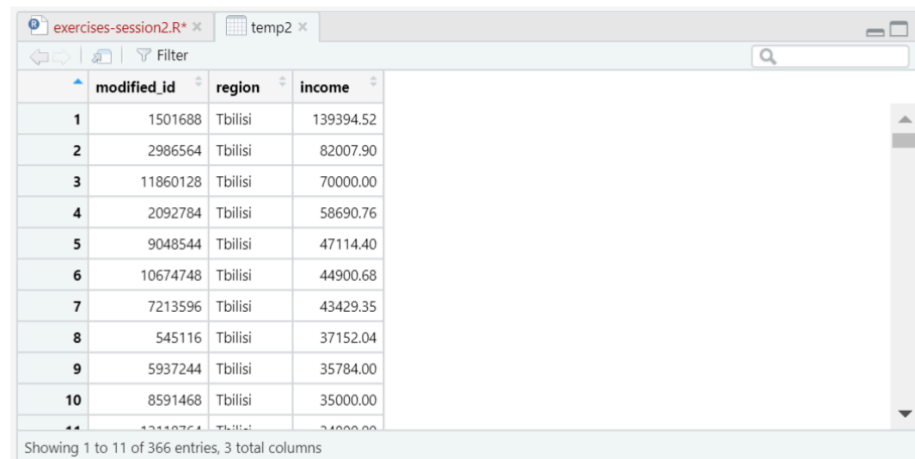
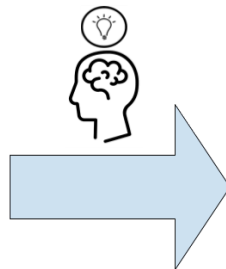
Use a função `arrange()` para ordenar. Por padrão, a ordenação no R é crescente. Incluímos o símbolo de menos (-) antes de `vprod` para ordenar de forma decrescente:

```
temp2 <- arrange(temp1, -vprod)
```



	modified_id	region	income
1	11539816	Tbilisi	3610.00
2	10763744	Tbilisi	29.00
3	10490076	Tbilisi	273.00
4	244516	Tbilisi	412.44
5	9048544	Tbilisi	47114.40
6	11783480	Tbilisi	5960.00
7	7770192	Tbilisi	925.00
8	3681488	Tbilisi	3320.00
9	4513508	Tbilisi	888.50
10	8282840	Tbilisi	3935.00

Showing 1 to 11 of 366 entries, 3 total columns



	modified_id	region	income
1	1501688	Tbilisi	139394.52
2	2986564	Tbilisi	82007.90
3	11860128	Tbilisi	70000.00
4	2092784	Tbilisi	58690.76
5	9048544	Tbilisi	47114.40
6	10674748	Tbilisi	44900.68
7	7213596	Tbilisi	43429.35
8	545116	Tbilisi	37152.04
9	5937244	Tbilisi	35784.00
10	8591468	Tbilisi	35000.00

Showing 1 to 11 of 366 entries, 3 total columns

- Observations: Business reported income of 2019, only for Tbilisi

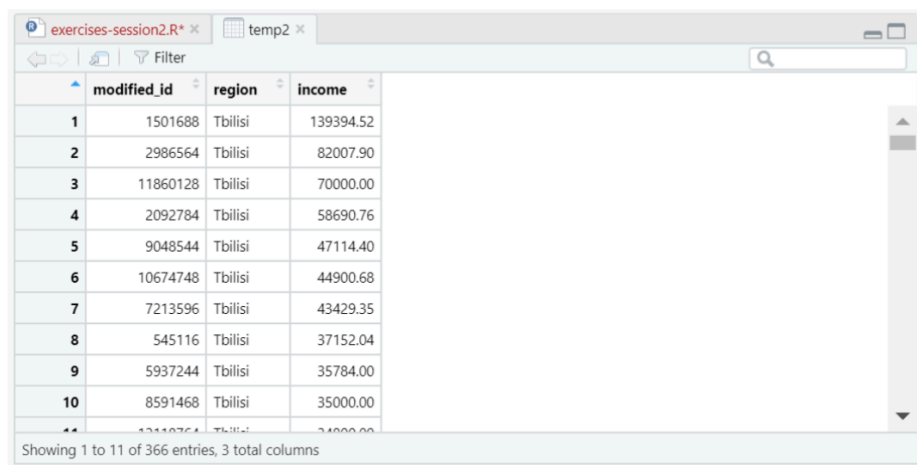
- Observations: Business reported income of 2019, only for Tbilisi, sorted descending by income

Filtragem e Classificação

3. Manter apenas os 50 primeiros itens após a ordenação

Use `filter()` novamente com o comando auxiliar `row_number()`

```
result_scenario1 <- filter(temp2, row_number() <= 50)
```

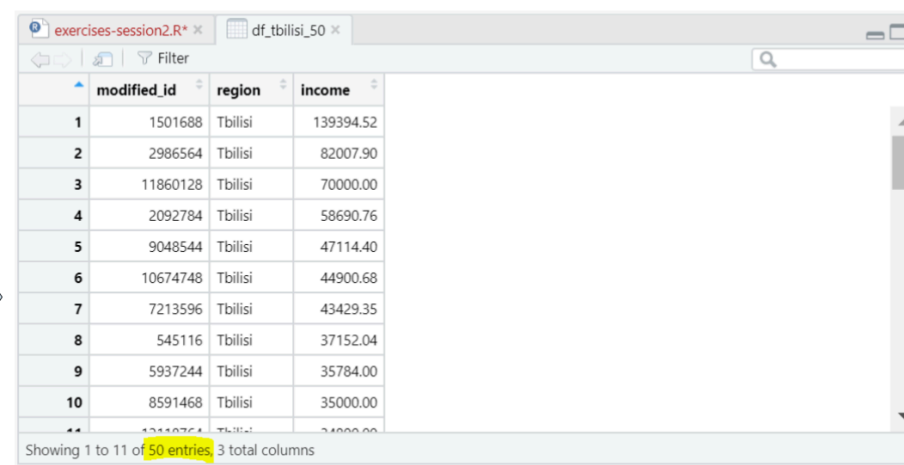
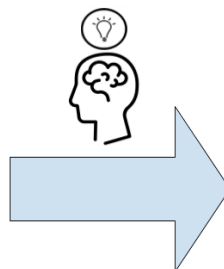


The RStudio interface shows a data frame named 'temp2' with 3 columns: 'modified_id', 'region', and 'income'. The data is sorted by income in descending order. The first 10 rows are visible, showing businesses from Tbilisi with various income levels.

	modified_id	region	income
1	1501688	Tbilisi	139394.52
2	2986564	Tbilisi	82007.90
3	11860128	Tbilisi	70000.00
4	2092784	Tbilisi	58690.76
5	9048544	Tbilisi	47114.40
6	10674748	Tbilisi	44900.68
7	7213596	Tbilisi	43429.35
8	545116	Tbilisi	37152.04
9	5937244	Tbilisi	35784.00
10	8591468	Tbilisi	35000.00

Showing 1 to 11 of 366 entries, 3 total columns

- Observations: Business reported income of 2019, only for Tbilisi, sorted descending by income



The RStudio interface shows a data frame named 'df_tbilisi_50' with 3 columns: 'modified_id', 'region', and 'income'. The data is sorted by income in descending order. The first 10 rows are visible, showing businesses from Tbilisi with various income levels. The status bar indicates 50 entries.

	modified_id	region	income
1	1501688	Tbilisi	139394.52
2	2986564	Tbilisi	82007.90
3	11860128	Tbilisi	70000.00
4	2092784	Tbilisi	58690.76
5	9048544	Tbilisi	47114.40
6	10674748	Tbilisi	44900.68
7	7213596	Tbilisi	43429.35
8	545116	Tbilisi	37152.04
9	5937244	Tbilisi	35784.00
10	8591468	Tbilisi	35000.00

Showing 1 to 11 of 50 entries, 3 total columns

- Observations: Business reported income of 2019, only for Tbilisi, sorted descending by income, and the 50 highest

Filtragem e Classificação

Exercício 5: Filtre e classifique seus dados

Agora que identificamos o formato do dataframe resultante e como obtê-lo, podemos escrever o código para isso.

1.- Seleção de linhas:

```
temp1 <- filter(base_nfe, mes_emissao == 10)
```

2.- Ordenação decrescente pelo valor do produto:

```
temp2 <- arrange(temp1, -vprod)
```

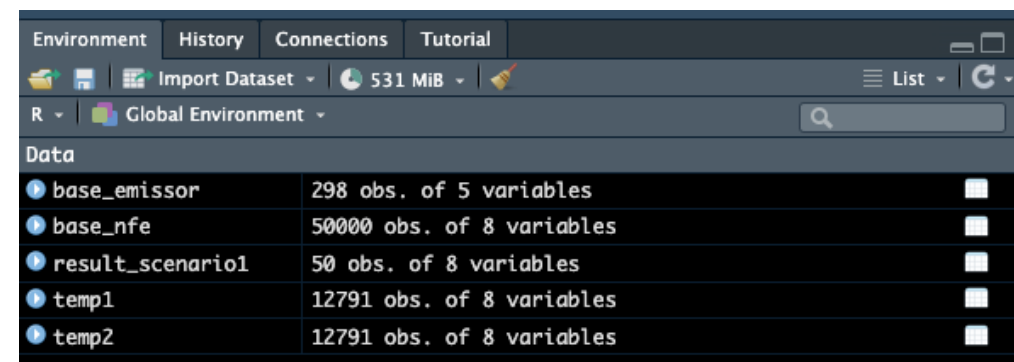
3.- Manter apenas os 50 primeiros itens:

```
result_scenario1 <- filter(temp2, row_number() <= 50)
```

Filtragem e Classificação

Algumas notas:

- `filter()`, `arrange()`, e `row_number()` são funções do `dplyr`. Lembre-se de carregar `dplyr` primeiro com `library(dplyr)`
 - Criamos dois dataframes intermediários: `temp1` e `temp2`
 - Podemos evitar isso usando o operador pipe (`%>%`), muito comum no R, mas não abordado nesta sessão
 - O dataframe resultante é `result_scenario1`



The screenshot shows the RStudio Environment pane for the Global Environment. It lists five data objects with their respective dimensions:

Data	
base_emissor	298 obs. of 5 variables
base_nfe	50000 obs. of 8 variables
result_scenario1	50 obs. of 8 variables
temp1	12791 obs. of 8 variables
temp2	12791 obs. of 8 variables

Filtragem e Classificação

Você pode verificar o resultado com `View(result_scenario1)`. Agora este dataframe contém exatamente o que precisávamos!



	nfuid_anon	dt_emissao	mes_emissao	emitcnpj8_anon	xprod	vprod	vicms	cfop
1	10000000631	2024-10-14	10	34942603	VITAMINA D 50.000UI X10 CAP	178517.50	4915.35	6102
2	10000000631	2024-10-14	10	34942603	VITAMINA D 50.000UI X04 CAP	132412.00	3645.87	6102
3	10000021140	2024-10-25	10	34942603	VITAMINA D 50.000UI X10 CAP	99969.80	2752.60	6102
4	10000000631	2024-10-14	10	34942603	VITAMINA D 7.000UI X30 CAP	96706.40	3057.22	6102
5	10000000631	2024-10-14	10	34942603	VITAMINA D 50.000UI X08 CAP	87013.50	4259.29	6102
6	10000021140	2024-10-25	10	34942603	VITAMINA D 7.000UI X30 CAP	82891.20	2620.47	6102
7	10000021140	2024-10-25	10	34942603	VITAMINA D 10.000UI X30 CAP	81450.60	1287.47	6102
8	10000000631	2024-10-14	10	34942603	VITAMINA D 10.000UI X30 CAP	74046.00	1170.42	6102
9	10000000631	2024-10-14	10	34942603	VITAMINA D 5.000UI X30 CAP	65643.20	1071.07	6102
10	10000021140	2024-10-25	10	34942603	VITAMINA D 5.000UI X30 CAP	65643.20	1071.07	6102
11	10000029922	2024-10-04	10	34942603	ADDERA D3 2000UI FR 90 COM REV	54868.80	6584.26	6106
12	10000015018	2024-10-28	10	87827638	ALTAD 15000UI 4CAP NV	54444.60	5552.19	6101
13	10000000631	2024-10-14	10	34942603	VITAMINA D 15.000UI X10 CAP	49372.40	2202.78	6102
14	10000021140	2024-10-25	10	34942603	VITAMINA D 15.000UI X10 CAP	42319.20	1888.10	6102
15	10000022299	2024-10-03	10	34942603	VITAMINA D3 15000UI CAP X 4	39503.52	4740.42	6102
16	10000021140	2024-10-25	10	34942603	VITAMINA D 50.000UI X04 CAP	33103.00	911.47	6102
17	10000030623	2024-10-03	10	87827638	ALTAD 15000UI 10CAP	32262.56	3290.10	6102
18	10000030623	2024-10-03	10	87827638	ALTAD 15000UI 4CAP NV	27827.24	2837.79	6101
19	10000015487	2024-10-11	10	34942603	ADDERA+IMUNIDADE MAX FR X 30CPRV	25556.40	3066.77	6106
20	10000027809	2024-10-23	10	34942603	NQ VITAMINA D3 50.000 UI X 8 CAP MOLE	25452.00	3054.24	6106
21	10000022299	2024-10-03	10	34942603	VITAMINA D3 7000UI CAP X 30	24529.34	2943.52	6102
22	10000021140	2024-10-25	10	34942603	VITAMINA D 10.000UI X08 CAP	24141.60	1413.64	6102
23	10000023570	2024-10-04	10	32575562	INPRUV D 50000UI 8CPR SN	22036.16	1983.25	6102
24	10000032181	2024-10-26	10	31668732	ALTAD CAPS 7.000UI C/30CP MOLES-OUTROS	20502.00	2460.24	6102
25	10000040112	2024-10-14	10	76620288	VITAMINA D 50.000UI 4CAPS	18620.00	2156.02	6102

Showing 1 to 25 of 50 entries, 8 total columns

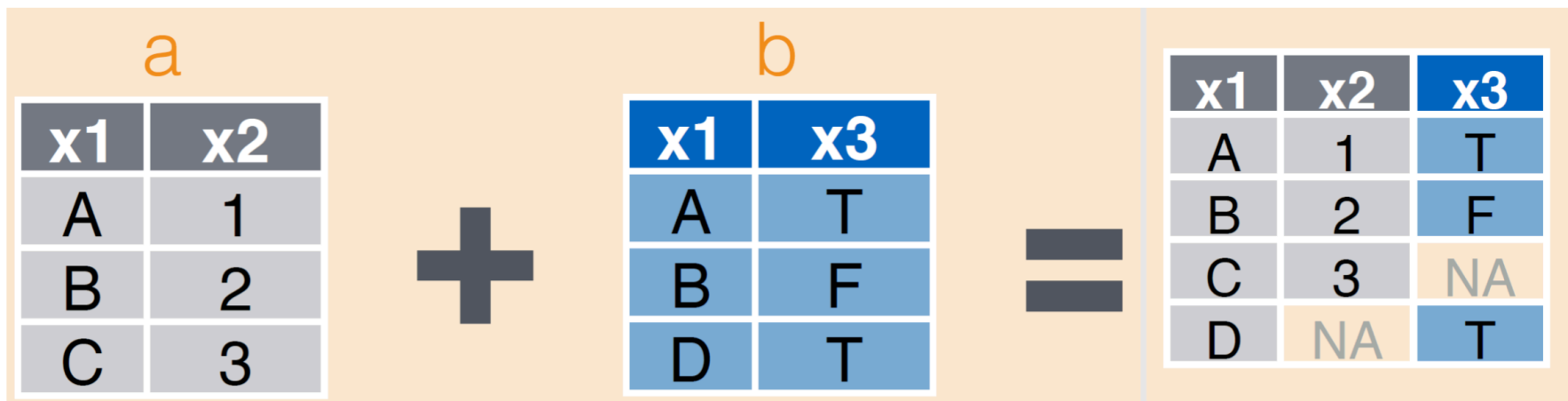
Filtragem e Classificação

- Filtragem e classificação são operações comuns na manipulação de dados no R
- Agora veremos uma nova operação muito útil: **junção de dados** (merging)

Junção de Dataframes

Junção de Dataframes

- Exploraremos mais uma operação comum na manipulação de dados: junção
- A junção de dados é usada para combinar colunas de um dataframe com outro
- Para isso, utilizamos uma "chave" que identifica as mesmas unidades nos diferentes dataframes



Nota: Imagem retirada do guia de manipulação de dados do RStudio.

Junção de Dataframes

Pedido de Análise de dados

Cenário 2:

"Nós precisamos do valor total de ICMS das NFes somente pra emissores de fora do Estado. Você talvez queira utilizar a nova base de dados no nível do emissor para este exercício"

Junção de Dataframes

Pedido de Análise de dados

Esses são os passos que precisamos seguir:

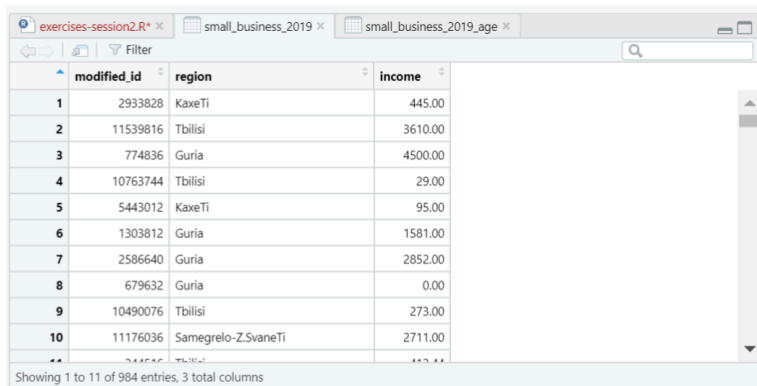
1. Selecionar as variáveis relevantes da `base_nfe`
2. Juntar as bases de dados
3. Filtrar somente emissores de fora do RS
4. Calcular o valor total de ICMS dessas notas

Junção de Dataframes

1. Selecionar as variáveis relevantes da `base_nfe`

Use `select()` pra esse passo:

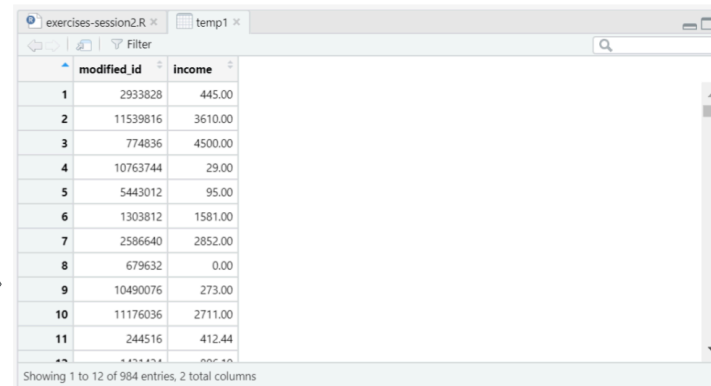
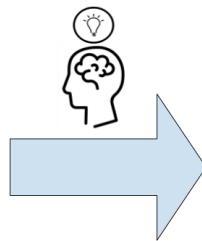
```
temp1 <- select(base_nfe, nfluid_anon, emitcnpj8_anon, vicms)
```



	modified_id	region	income
1	2933828	KaxeTi	445.00
2	11539816	Tbilisi	3610.00
3	774836	Guria	4500.00
4	10763744	Tbilisi	29.00
5	5443012	KaxeTi	95.00
6	1303812	Guria	1581.00
7	2586640	Guria	2852.00
8	679632	Guria	0.00
9	10490076	Tbilisi	273.00
10	11176036	Samegrelo-Z.SvaneTi	2711.00

Showing 1 to 11 of 984 entries, 3 total columns

- Observations: Business reported income and region of 2019
- Columns: `modified_id`, `region`, `income`



	modified_id	income
1	2933828	445.00
2	11539816	3610.00
3	774836	4500.00
4	10763744	29.00
5	5443012	95.00
6	1303812	1581.00
7	2586640	2852.00
8	679632	0.00
9	10490076	273.00
10	11176036	2711.00
11	244516	412.44

Showing 1 to 12 of 984 entries, 2 total columns

- Observations: Business reported income in 2019
- Columns: `modified_id`, `income`

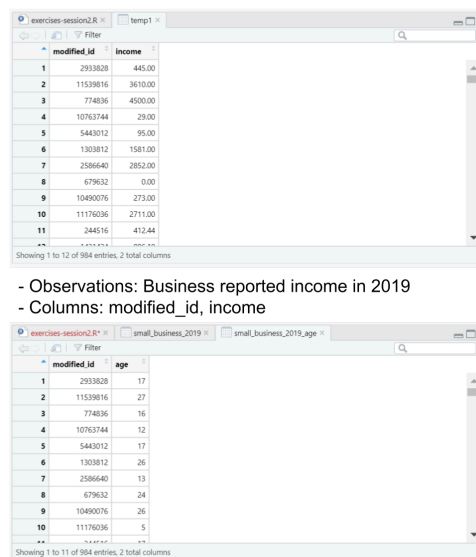
Junção de Dataframes

2. Juntar as bases de dados

Use `left_join()` pra juntar as bases:

```
temp2 <- left_join(temp1, base_emissor, by = "emitcnnpj8_anon")
```

- Os dois primeiros argumentos são as bases que queremos juntar
- O terceiro argumento (com um nome) é a variável-chave que conecta as bases



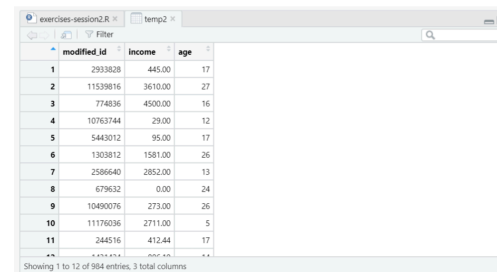
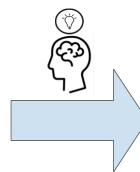
The screenshot shows two data frames in RStudio. The top window, titled 'temp1', displays a table with columns 'modified_id' and 'income'. The bottom window, titled 'small_business_2019_age', displays a table with columns 'modified_id' and 'age'.

	modified_id	income
1	2933828	445.00
2	11539816	3610.00
3	774836	4500.00
4	10763744	29.00
5	5443012	95.00
6	1303812	1581.00
7	2586640	2852.00
8	679632	0.00
9	10490076	273.00
10	11176036	2711.00
11	244516	412.44

- Observations: Business reported income in 2019
- Columns: modified_id, income

	modified_id	age
1	2933828	17
2	11539816	27
3	774836	16
4	10763744	12
5	5443012	17
6	1303812	26
7	2586640	13
8	679632	24
9	10490076	26
10	11176036	5

- Observations: Business age in 2019
- Columns: modified_id, age



The screenshot shows the resulting data frame 'temp2' in RStudio. It contains columns 'modified_id', 'income', and 'age'.

	modified_id	income	age
1	2933828	445.00	17
2	11539816	3610.00	27
3	774836	4500.00	16
4	10763744	29.00	12
5	5443012	95.00	17
6	1303812	1581.00	26
7	2586640	2852.00	13
8	679632	0.00	24
9	10490076	273.00	26
10	11176036	2711.00	5
11	244516	412.44	17

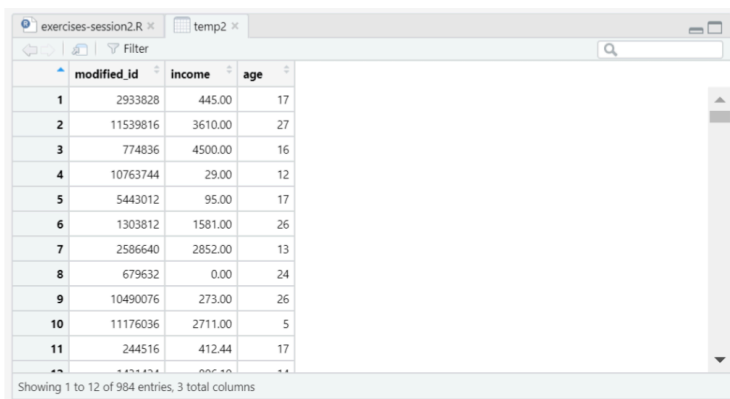
- Observations: Business age and reported income in 2019
- Columns: modified_id, income, age

Junção de Dataframes

3. Filtrar somente emissores de fora do RS

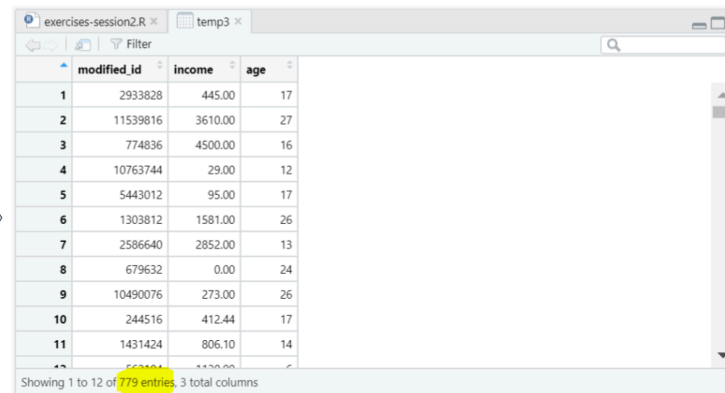
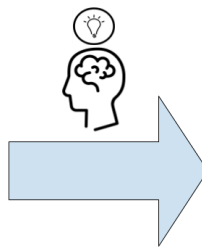
Use `filter()` outra vez:

```
temp3 <- filter(temp2, emituf != "RS")
```



	modified_id	income	age
1	2933828	445.00	17
2	11539816	3610.00	27
3	774836	4500.00	16
4	10763744	29.00	12
5	5443012	95.00	17
6	1303812	1581.00	26
7	2586640	2852.00	13
8	679632	0.00	24
9	10490076	273.00	26
10	11176036	2711.00	5
11	244516	412.44	17

Showing 1 to 12 of 984 entries, 3 total columns



	modified_id	income	age
1	2933828	445.00	17
2	11539816	3610.00	27
3	774836	4500.00	16
4	10763744	29.00	12
5	5443012	95.00	17
6	1303812	1581.00	26
7	2586640	2852.00	13
8	679632	0.00	24
9	10490076	273.00	26
10	244516	412.44	17
11	1431424	806.10	14

Showing 1 to 12 of 779 entries, 3 total columns

- Observations: Business age and reported income in 2019
- Columns: modified_id, income, age

- Observations: Business age and reported income in 2019, only for firms with more than five years of age
- Columns: modified_id, income, age

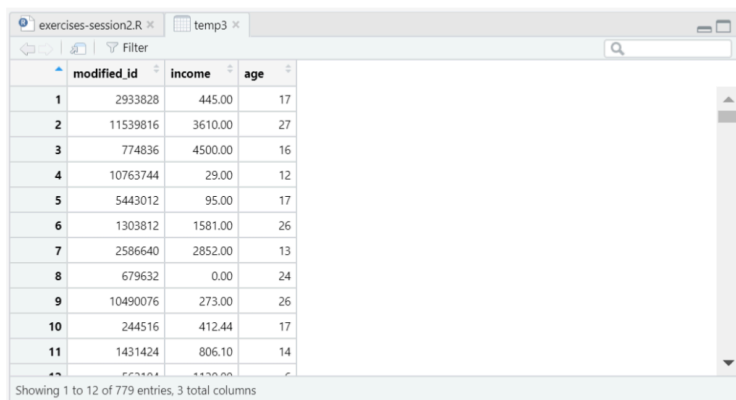
Junção de Dataframes

4. Calcular o valor total de ICMS dessas notas

Use `summarise()` e `sum()`.

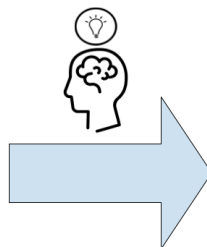
- `summarise()` nos permite agregar os dados usando uma função; nesse caso a soma (`sum()`)

```
total_income <- summarise(temp3, sum(vicms))
```



	modified_id	income	age
1	2933828	445.00	17
2	11539816	3610.00	27
3	774836	4500.00	16
4	10763744	29.00	12
5	5443012	95.00	17
6	1303812	1581.00	26
7	2586640	2852.00	13
8	679632	0.00	24
9	10490076	273.00	26
10	244516	412.44	17
11	1431424	806.10	14

- Observations: Business age and reported income in 2019, only for firms with more than five years of age
- Columns: modified_id, income, age



2592266

- Total business income for small businesses with more than five years in 2019

Junção de Dataframes

Exercício 6: Juntando as bases de dados

Aplique os passos discutidos anteriormente pra calcular o valor total de ICMS nas NFes emitidas fora do RS

1. Selecionar as variaveis relevantes da `base_nfe`

```
temp1 <- select(base_nfe, nfuid_anon, emitcnpj8_anon, vicms)
```

- 2.- Juntar as bases de dados

```
temp2 <- left_join(temp1, base_emissor, by = "emitcnpj8_anon")
```

- 3.- Filtrar somente emissores de fora do RS

```
temp3 <- filter(temp2, emituf != "RS")
```

1. Calcular o valor total de ICMS dessas notas

```
total_income <- summarise(temp3, sum(vicms))
```

Junção de Dataframes

`total_income` é uma base de dados com uma observação e o valor que buscamos

```
print(total_income)
```

```
##    sum(vicms)
```

```
## 1    254145.5
```

Junção de Dataframes

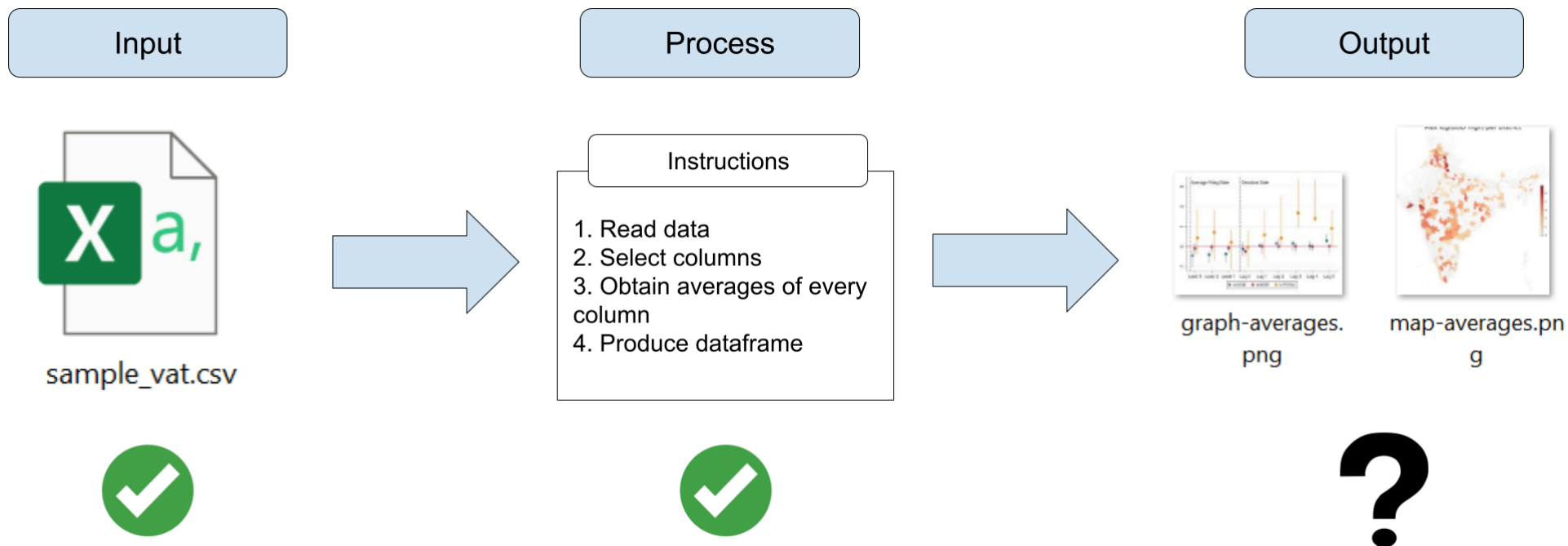
Esse foram alguns exemplos de como limpar dados. Outras operações comuns incluem:

Operação	Função no <code>dplyr</code>
Selecionar colunas	<code>select()</code>
Filtrar linhas (com base em uma condição)	<code>filter()</code>
Criar novas colunas	<code>mutate()</code>
Criar novas colunas com base em uma condição	<code>mutate()</code> e <code>case_when()</code>
Adicionar novas linhas	<code>add_row()</code>
Mesclar dataframes	<code>inner_join()</code> , <code>left_join()</code> , <code>right_join()</code> , <code>full_join()</code>
Empilhar dataframes	<code>bind_rows()</code>
Remover duplicatas	<code>distinct()</code>
Agrupar e criar indicadores resumidos	<code>group_by()</code> , <code>summarize()</code>
Passar um resultado como primeiro argumento para a próxima função	<code>%>%</code> (operador, não função)

Exportando resultados

Exportando resultados

- Até agora, vimos exemplos das partes 1 e 2 do trabalho com dados
- E como exportar resultados?



- Vamos ver no próximo exercício

Exportando resultados

Exportando dataframes

- A maneira mais fácil de exportar um dataframe ou número é com a função `write.csv()`
- `write.csv()` cria um arquivo CSV com o dataframe
- Ele recebe dois argumentos básicos:
 1. O nome do objeto que você deseja exportar
 2. Um caminho de arquivo para exportar o objeto
- `write.csv()` inclui os números das linhas por padrão. Você pode adicionar o argumento `row.names = FALSE` para evitar isso

Exportando resultados

Exercício 7: Exporte `result_scenario1` e `total_income`

1. Use este código para exportar os resultados dos dois últimos exercícios:

```
write.csv(result_scenario1,  
          "result_scenario1.csv",  
          row.names = FALSE)  
  
write.csv(total_income,  
          "total_income.csv",  
          row.names = FALSE)
```

Exportando resultados

Agora `result_scenario1.csv` e `total_income.csv` devem aparecer no seu computador (provavelmente no folder `Documents`).

Name	Date modified [^]	Type	Size
 .Rproj.user	9/19/2023 2:37 AM	File folder	
 1-introduction-to-r_cache	9/19/2023 3:30 AM	File folder	
 img	9/19/2023 3:43 PM	File folder	
 2-data-wrangling_cache	9/19/2023 3:46 PM	File folder	
 data	9/20/2023 12:11 AM	File folder	
 libs	9/20/2023 5:09 AM	File folder	
 .Rhistory	9/19/2023 4:20 PM	RHISTORY File	1 KB
 1-introduction-to-r.Rmd	9/20/2023 1:34 AM	RMD File	24 KB
 1-introduction-to-r.html	9/20/2023 1:34 AM	Chrome HTML Docu...	30 KB
 exercises-session1.R	9/20/2023 1:41 AM	R File	1 KB
 202309.Rproj	9/20/2023 1:58 AM	R Project	1 KB
 exercises-session2.R	9/20/2023 4:37 AM	R File	1 KB
 2-data-wrangling.Rmd	9/20/2023 5:09 AM	RMD File	25 KB
 2-data-wrangling.html	9/20/2023 5:09 AM	Chrome HTML Docu...	30 KB
 total_income.csv	9/20/2023 5:11 AM	Microsoft Excel Com...	1 KB
 df_tbilisi_50.csv	9/20/2023 5:12 AM	Microsoft Excel Com...	2 KB

Exportando resultados

Comentários sobre caminhos (*paths*)

- O segundo argumento de `write.csv()` especifica o caminho do arquivo que vamos exportar

```
write.csv(result_scenariol,  
          "result_scenariol.csv",  
          row.names = FALSE)
```

- Você pode incluir qualquer caminho no seu computador e o R vai salvar nessa localização
 - Por exemplo: `"/Users/tscot/Downloads"` exporta os resultados pra pasta de *downloads* do meu computador (isso não funcionaria em outras máquinas)
 - Note que o caminho no R usa barra a direita (*forward slashes*) (`/`). Barra invertida (*back slash*) (`\`) **do not work in R**

Exportando resultados

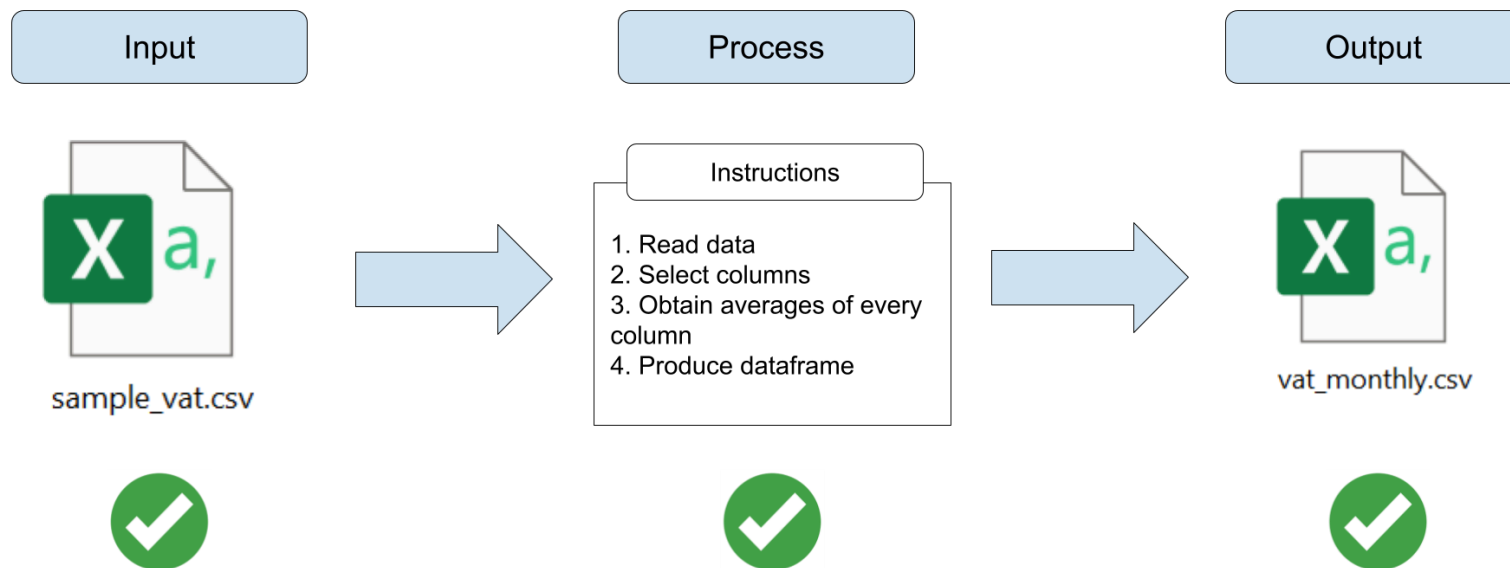
Comentários sobre caminhos (*paths*)

```
write.csv(result_scenariol1,  
          "result_scenariol1.csv",  
          row.names = FALSE)
```

- Se você só incluir o nome do arquivo, R vai exportar pra localização atual do RStudio - normalmente o folder `Documents`
- Você pode checar a localização atual do RStudio com a função `getwd()`

Exportando resultados

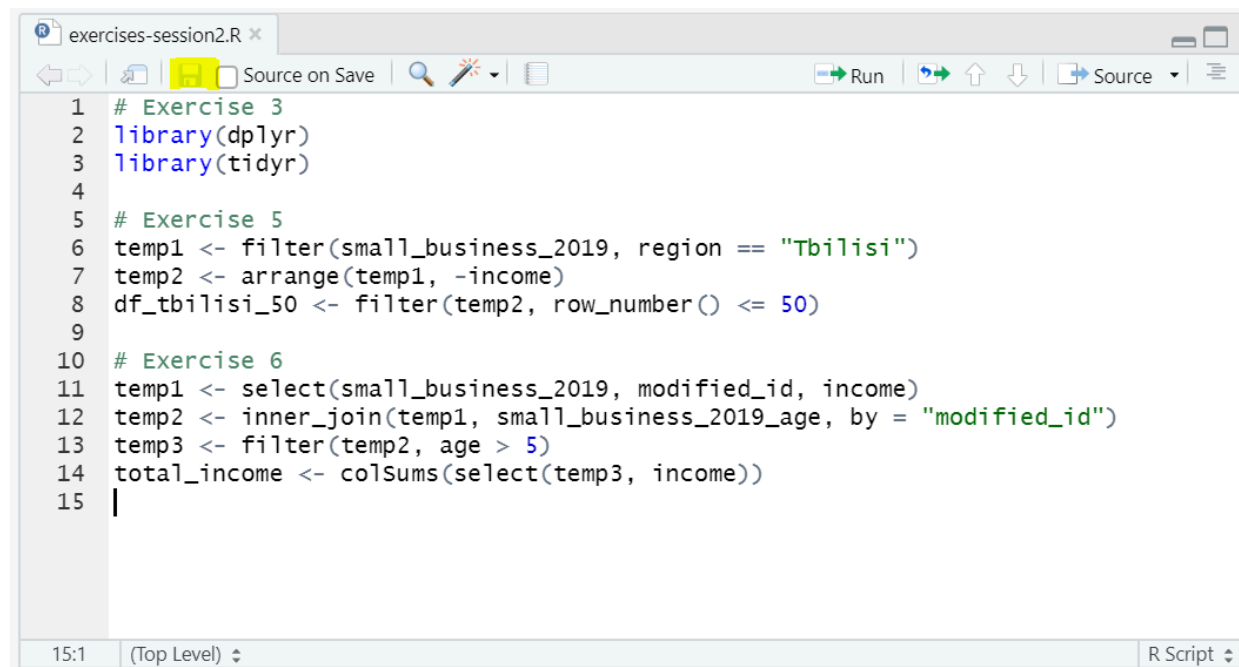
Nosso pipeline de dados agora está completo!!!



Finalizando

Finalizando

- Não se esqueça de salvar seu trabalho!
- Utilize `#` para adicionar comentários no código
- Salve seu arquivo em um local seguro



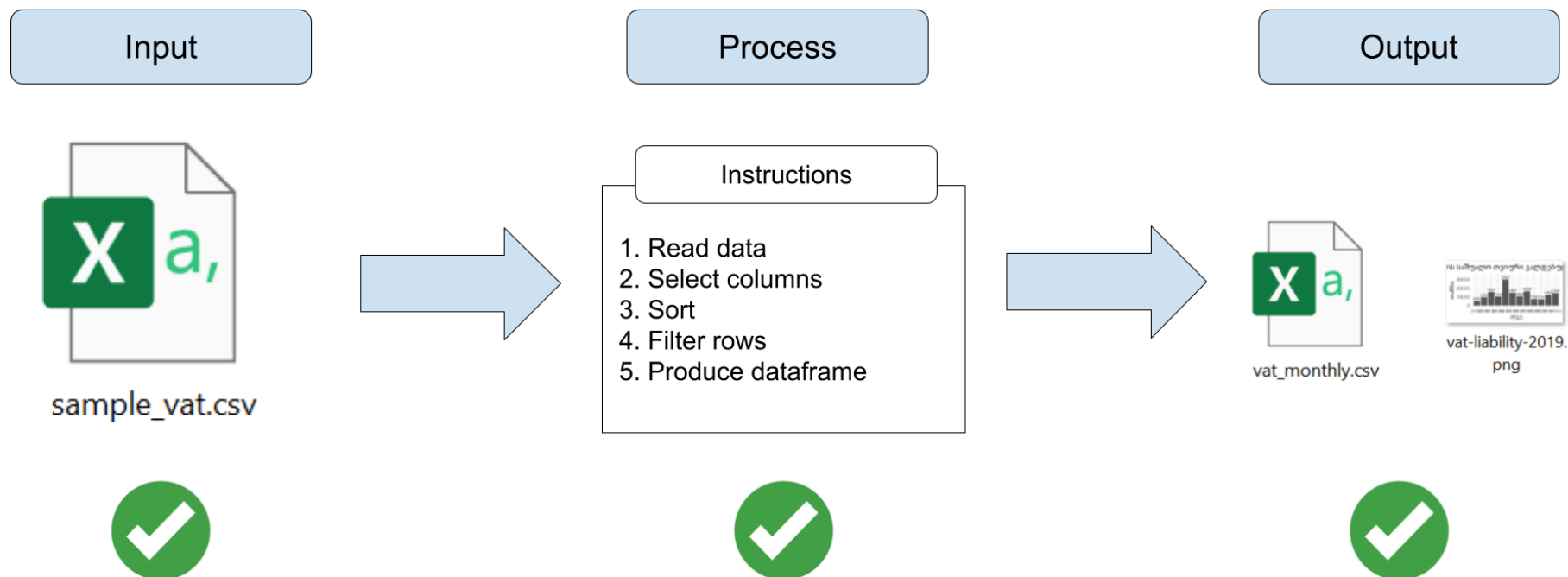
```
exercises-session2.R
Source on Save
Run
Source

1 # Exercise 3
2 library(dplyr)
3 library(tidyr)
4
5 # Exercise 5
6 temp1 <- filter(small_business_2019, region == "Tbilisi")
7 temp2 <- arrange(temp1, -income)
8 df_tbilisi_50 <- filter(temp2, row_number() <= 50)
9
10 # Exercise 6
11 temp1 <- select(small_business_2019, modified_id, income)
12 temp2 <- inner_join(temp1, small_business_2019_age, by = "modified_id")
13 temp3 <- filter(temp2, age > 5)
14 total_income <- colSums(select(temp3, income))
15 |

15:1 (Top Level) R Script
```

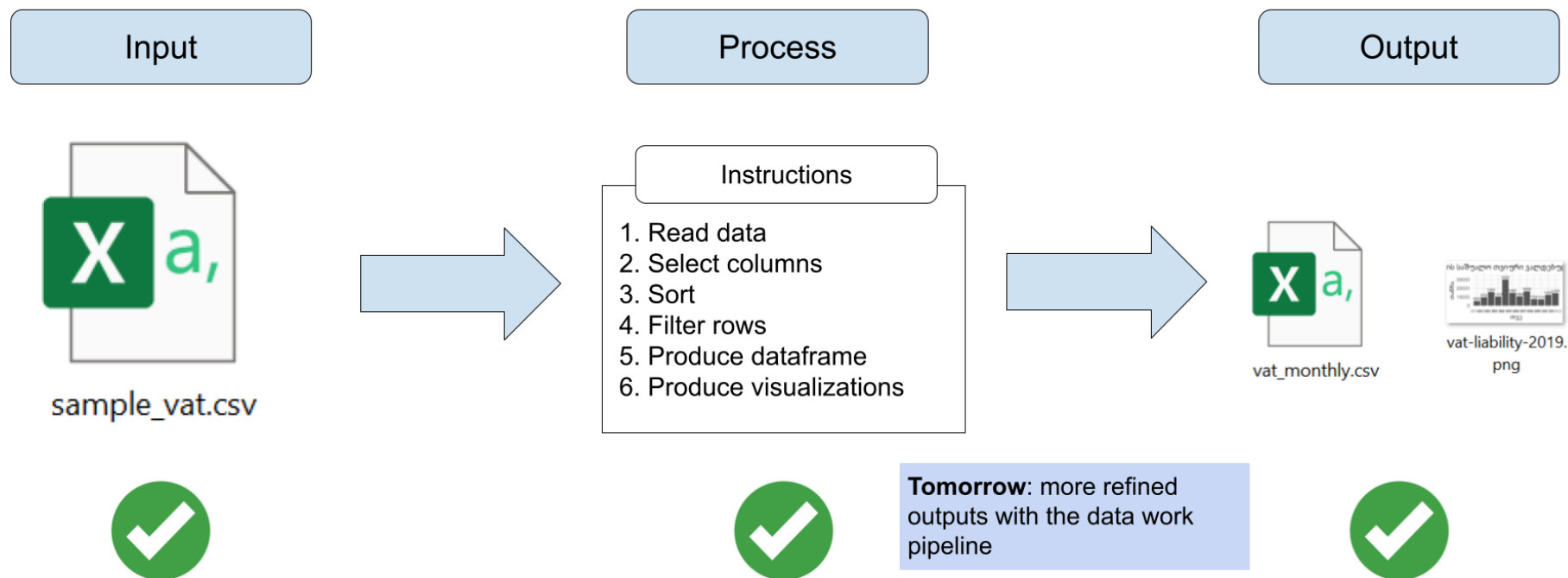
Conclusão

- Exploramos operações essenciais de manipulação de dados
- Aplicamos filtragem, ordenação, junção e exportação de resultados
- Essas técnicas são fundamentais para análise de dados no R



Finalizando

Pipeline de trabalho com dados



Apêndice

Agregação

Cenário: Imagine que você recebe a seguinte solicitação:

"Estamos preparando um relatório onde queremos incluir o valor total e médio por produto. Você pode calcular esses números?"

Agregação

A manipulação dos dados aqui envolve várias operações:

1. Manter apenas as colunas relevantes e descartar as demais
2. Agrupar o dataframe por produto
3. Calcular o valor total por produto
4. Calcular o valor médio por produto

A operação de transformar um dataframe com um nível mais baixo de observações (exemplo: NFE) para um nível agregado (exemplo: produto) é chamada de **agregação**.

1. Manter apenas as colunas relevantes

Use `select()` para isso:

```
temp1 <- select(base_nfe, xprod, vprod)
```

2. Agrupar por produto

Use `group_by()`:

```
temp2 <- group_by(temp1, xprod)
```

3. Calcular as colunas agregadas: valor total e médio

Use `summarize()` combinado com `sum()` e `mean()`:

```
product_df <- summarize(temp2,  
                        total = sum(vprod),  
                        average = mean(vprod))
```

Exercício: Agregue seus dados

Agora podemos escrever o código para executar a agregação de dados.

1. Seleção de colunas: `temp1 <- select(base_nfe, xprod, vprod)`
2. Agrupamento por produto: `temp2 <- group_by(temp1, xprod)`
3. Cálculo do total e da média por produto:

```
product_df <- summarize(temp2,  
                        total = sum(vprod),  
                        average = mean(vprod))
```

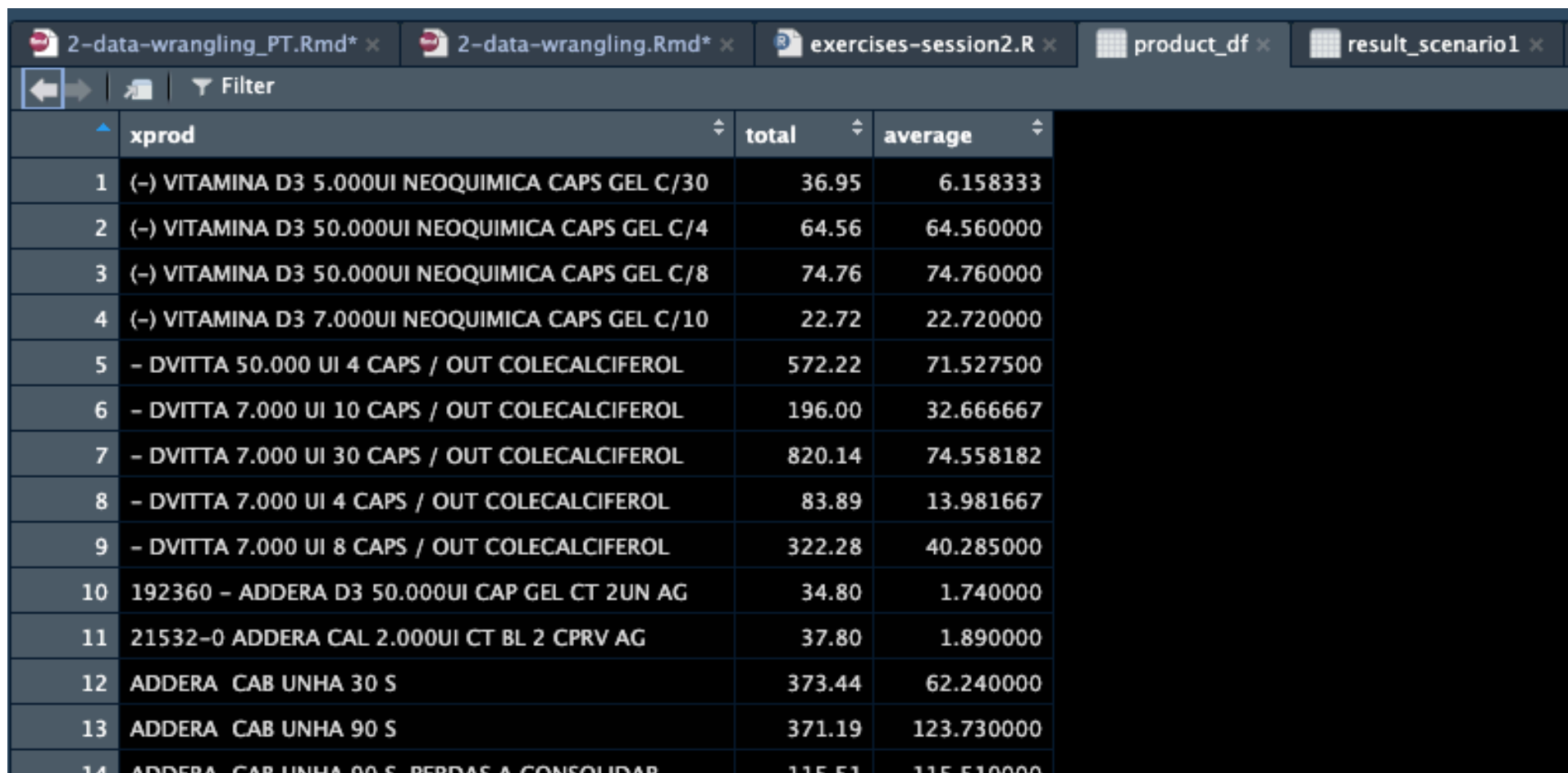
Algumas observações:

- Lembre-se de que todas essas funções fazem parte do `dplyr`
- Há uma quebra de linha e um espaço de tabulação entre cada argumento de `summarize()`. Isso é utilizado para maior clareza no código, mas o R ignora essas quebras de linha e espaços quando usados entre os argumentos das funções

Apêndice

Agregação

Você pode verificar o resultado com `View(product_df)`. Agora seus dados estão agregados exatamente como necessário!



The screenshot shows an RStudio window with several tabs open: '2-data-wrangling_PT.Rmd*', '2-data-wrangling.Rmd*', 'exercises-session2.R', 'product_df', and 'result_scenario1'. The 'product_df' tab is active, displaying a data table with three columns: 'xprod', 'total', and 'average'. The table contains 14 rows of data, each representing a different product and its associated values.

	xprod	total	average
1	(-) VITAMINA D3 5.000UI NEOQUIMICA CAPS GEL C/30	36.95	6.158333
2	(-) VITAMINA D3 50.000UI NEOQUIMICA CAPS GEL C/4	64.56	64.560000
3	(-) VITAMINA D3 50.000UI NEOQUIMICA CAPS GEL C/8	74.76	74.760000
4	(-) VITAMINA D3 7.000UI NEOQUIMICA CAPS GEL C/10	22.72	22.720000
5	- DVITTA 50.000 UI 4 CAPS / OUT COLECALCIFEROL	572.22	71.527500
6	- DVITTA 7.000 UI 10 CAPS / OUT COLECALCIFEROL	196.00	32.666667
7	- DVITTA 7.000 UI 30 CAPS / OUT COLECALCIFEROL	820.14	74.558182
8	- DVITTA 7.000 UI 4 CAPS / OUT COLECALCIFEROL	83.89	13.981667
9	- DVITTA 7.000 UI 8 CAPS / OUT COLECALCIFEROL	322.28	40.285000
10	192360 - ADDERA D3 50.000UI CAP GEL CT 2UN AG	34.80	1.740000
11	21532-0 ADDERA CAL 2.000UI CT BL 2 CPRV AG	37.80	1.890000
12	ADDERA CAB UNHA 30 S	373.44	62.240000
13	ADDERA CAB UNHA 90 S	371.19	123.730000
14	ADDERA CAB UNHA 90 S PERDAS A CONSOLIDAR	115.51	115.510000